



Universidad Argentina de la Empresa
Facultad de Ingeniería y Ciencias Exactas

Programación I

Aula 509, Lima 1

Grupo 7

Hotel Boutique

Alumnos

Desarrollador Diego Mora - LU: 1224472

Desarrollador Ramiro Fiol - LU: 1139391

Analista Ignacio Uliczki - LU: 1238960

Jefe Analista Román Oro - LU 1210619

Analista Valentino Zaninetti - LU 1202296

Docentes

Escandell, Gustavo Emmanuel

Selles, Melinda Lujan

Link al repositorio Github: https://github.com/MoraDiego02/Proyecto_Hotel

Presentación del proyecto

Nuestro objetivo principal es desarrollar un sistema de gestión de reservas hoteleras. Todo comienza cuando el usuario ingresa su nombre, apellido, DNI, mail y número de teléfono. Luego selecciona la habitación, piso y ubicación que se va a hospedar. Una vez hecha la elección, el sistema mostrará una matriz donde el usuario podrá ver el piso y habitación a reservar. Luego el usuario deberá abonar el precio de la reserva la cual puede variar dependiendo del día los cuales algunos tienen descuento o recargo además de la forma de pago; si se realiza con tarjeta, tendrá un recargo del 5%. Luego el sistema entregará un comprobante detallando: Nombre y apellido, mail, DNI del usuario, piso reservado, habitación y monto final a pagar.

Alcance del proyecto

1. Toma de datos y validación inicial Todo arranca cuando el usuario quiere hacer una reserva. El sistema le va a pedir su DNI, el sistema contiene una lista la cual guarda los DNIs ingresados así no se pueden repetir en un futuro. el tipo de habitación y piso que busca además de la vista las cuales pueden ser al jardín o a la ciudad, también cómo prefiere pagar. Para asegurarnos de que no haya errores de tipeo o datos falsos, el sistema va a validar todos estos ingresos.

2. Asignación de habitaciones Una vez que el cliente está validado, pasamos a la gestión del hotel. Para representar visualmente los pisos y las habitaciones de forma eficiente, vamos a crear una matriz donde se podrá ver el piso y habitación seleccionada por el usuario.

3. Métodos de pago

El precio de la estadía no es igual en todos los casos, sino que el sistema lo calcula en el momento basándose en dos cosas:

- **El día de ingreso:** Si entran en la semana (lunes a jueves), se llevan un 15% de descuento. Si vienen el fin de semana (viernes a domingo), hay un recargo del 10%.
- **El medio de pago:** Si pagan en efectivo el precio queda igual, pero si usan tarjeta de crédito se suma un 5% extra.

Al confirmar la reserva, el sistema le genera al cliente un comprobante claro y detallado. Ahí va a poder ver sus datos, qué habitación y piso le tocó y el precio final.

4. Volver a reservar

En esta instancia el usuario podrá volver a realizar una reserva si así lo desea, si es así el programa se vuelve a ejecutar desde el inicio llamando a la función main(). Si no desea otra reserva el sistema se cerrará dejando un mensaje de agradecimiento por la reserva.

Para mantener un orden en el proyecto optamos por un desarrollo completamente modularizado. separamos las tareas en distintas funciones (una para cargar, otra para validar, etc.) logrando un código claro y reutilizable. Además, decidimos que todo el

programa va a correr dentro de una función principal llamada main ejecutándose con el clásico `if __name__ == "__main__": main()` al final de nuestro archivo Python.

En este apartado está la documentación de cada función del proyecto junto con la explicaciones de uso de:

- Listas avanzadas
- Matrices
- Funciones lambda
- Expresiones regulares

Funciones

Archivo funciones_hotel.py

- `ingresar_dni`: La función `ingresar_dni` solicita al usuario que ingrese su dni y valida que tenga 8 dígitos enteros. si el valor es correcto retorna el DNI ingresado.
- `gestion_reserva`: Esta función gestiona el cálculo del precio final de una reserva. genera un día aleatorio de la semana y aplica un descuento del 20% si este día se encuentra entre el lunes y jueves, o un recargo del 10% si es entre viernes y domingo. Luego solicita al usuario el método de pago (Efectivo o tarjeta) y aplica un recargo adicional del 5% si se selecciona tarjeta. El precio final se calcula usando *reduce* para combinar los multiplicadores y se retorna el monto total a abonar.
- `tipo_habitaciones`: La función muestra por pantalla los tipos de habitaciones que están disponibles. Usa *map* junto con *enumerate* para generar y formatear de forma automática la lista numerada de operaciones.
- `filtrar_ubicaciones_por_habitacion`: Esta función devuelve las ubicaciones disponibles según el tipo de habitación seleccionada. Usa *filter* para seleccionar solo aquellas ubicaciones que permiten la habitación indicada
- `pedir_piso`: Esta función solicita al usuario que ingrese el número de piso para la reserva. Valida que el valor esté entre 1 y 3, y en caso contrario vuelve a pedir el dato. Devuelve el número de piso ingresado
- `pedir_habitacion`: La función solicita al usuario que ingrese el número de habitación que desea. Valida que el valor esté entre 1 y 3, y en caso de error solicita nuevamente el ingreso. Retorna el número de habitación seleccionado.
- `Mostrar_habitaciones`: La función muestra en pantalla el estado de ocupación de las habitaciones organizadas por piso. Usa una matriz donde los valores indican si una habitación está ocupada o disponible, representando las que están ocupadas con una X. Devuelve la matriz.
- `cargar_matriz`: Esta función crea e inicializa una matriz de 3x3 con valores en 0, donde cada fila representa un piso y cada columna una habitación. Retorna la matriz creada.
- `comprobante_reserva`: La función muestra por pantalla un comprobante de la reserva realizada, incluyendo el DNI del cliente, el piso, la habitación seleccionada y el precio final a abonar.
- `inicio`: Esta función muestra un mensaje de bienvenida al usuario al iniciar el sistema
- `cargar_datos`: La función `cargar_datos` se encarga de solicitar al usuario su nombre, apellido, DNI, correo electrónico y número de teléfono. Cada dato ingresado es validado utilizando expresiones regulares para asegurar que cumpla con el formato esperado. Retorna los datos ingresados por el usuario.

- `solicitar_email`: La función `solicitar_email` solicita al usuario que ingrese su correo electrónico y valida el formato utilizando una expresión regular. Si el formato es inválido, se muestra un mensaje de error y se vuelve a solicitar el correo electrónico hasta que se ingrese uno válido.
- `habitaciones_hotel`: Devuelve un diccionario con la información de las habitaciones del hotel, organizado por pisos y número de habitación.
- `mostrar_detalle_eleccion`: Muestra por pantalla el tipo, vista y precio de la habitación elegida.
- `eleccion_habitacion`: solicita al usuario que elija un tipo de habitación
- `listado_habitaciones`: Muestra un listado completo de las habitaciones del hotel, organizado por piso, tipo y precio base. Controla que el diccionario tenga la estructura esperada.
- `registrar_checkin`: Genera una fecha de check-in aleatoria, con el formato (día, mes, año, hora, minuto, día de la semana). El día de la semana se recyle como parámetro para que coincida con el día generado en `gestion_reserva()`.
- `guardarReservaHistorial`: Agrega una línea nueva al archivos `HotelHistorialReservas.csv` con los datos de la reserva, sin pisar las reservas ya guardadas. "reserva" es la tupla (nombre, apellido, dni, mail, telefono, piso, hab, precio_final, checkin)
- `leerHistorial`: Lee `HotelHistorialReservas.csv` línea por línea y devuelve la lista de reservas en el mismo formato de la tupla que se usa en memoria: (nombre, apellido, dni, mail, telefono, piso, hab, precio_final, checkin) Si el archivo no existe o está vacío, devuelve una lista vacía.
- `guardarEstadoHabitaciones`: Reescribe por completo `HotelHistorialReservas.csv` con el estado actual de la matriz del hotel (una línea por habitación: id;piso;hab;ocupada). El id es "piso-hab" para identificar cada habitación de forma única.
- `actualizarEstadoHabitacion`: Marca una única habitación como ocupada dentro de `HotelEstadoHabitacion.csv`, primero copia el archivo viejo a uno temporal, luego se reescribe el archivo original línea por línea aplicando el cambio, y por último se vacía el temporal.
- `leerEstadoHabitaciones`: Lee `HotelEstadoHabitacion.csv` y reconstruye la matriz del hotel (1 = ocupada, " " = libre, igual que `cargar_matriz`). Si el archivo existe pero está vacío (recién creado a mano), se lo completa con todas las habitaciones libres antes de devolver la matriz, ya que `actualizarEstadoHabitacion()` necesitaba encontrar cada línea para poder modificarla.

Archivo `Hotel_proy.py`

- `main`: La función principal del programa. Se encarga de coordinar el flujo general del sistema de reservas, incluyendo dentro la carga de datos del cliente, la selección de piso, habitación y ubicación, la actualización del estado de las habitaciones, el cálculo del precio final y la generación del comprobante de reserva.

Lambda: Se hizo uso de una función lambda para definir una función que genere un número aleatorio entre el 0 y 6. Se asignó esta función a la variable `generar_dia` y luego se invoca como una función común para conseguir un valor que represente un día de la semana.

Matrices: En este proyecto utilizamos las listas bidimensionales mejor conocidas como matrices para representar la ubicación del piso y habitación de la reserva de los usuarios. Esto es visible ya que se marcará con una X la opción elegida previamente.

Diccionarios: Se utilizó un diccionario anidado para representar la información del hotel mediante la función `habitaciones_hotel()`. La clave principal corresponde al número de piso y su valor es, a su vez, otro diccionario donde la clave es el número de habitación y el valor es un tercer diccionario con los detalles de esa habitación (tipo, descripción y precio). Esta estructura permite acceder de forma directa a los datos de cualquier habitación indicando primero el piso y luego el número de habitación, por ejemplo `datos_hotel[1][3]["precio"]`, sin necesidad de recorrer listas para encontrar la información.

Tuplas: Se utilizaron tuplas para almacenar información que no debía modificarse una vez generada. La función `registrar_checkin()` arma una tupla `checkin` con el día, mes, año, hora, minuto y día de la semana de la fecha de ingreso, datos que quedan fijos desde el momento en que se confirma la reserva. De la misma forma, cada reserva completa (`nueva_reserva`) se guarda como una tupla con los datos del cliente, la habitación elegida, el precio final y el propio `checkin`, antes de agregarse al historial y guardarse en el archivo. Al ser tuplas, se garantiza que estos datos no puedan alterarse accidentalmente en otra parte del programa.

Excepciones: Se utilizó la estructura `try/except` en todas las validaciones de datos del usuario (nombre, DNI, mail, teléfono, piso, habitación y método de pago), de manera que si el dato ingresado no cumple con el formato esperado se lanza un `ValueError` y se vuelve a solicitar el dato dentro de un bucle, evitando que el programa se interrumpa. También se utilizó para controlar que una habitación seleccionada no esté ya ocupada, lanzando una excepción cuando corresponde y pidiendo al usuario que elija otra. Por último, se aplicó `try/except` con `IOError` y `OSError` en todas las funciones que abren o escriben archivos (como `guardarReservaHistorial()`, `leerHistorial()` y `actualizarEstadoHabitacion()`), para que un eventual error al acceder a los archivos no detenga la ejecución del sistema.

Conjuntos: Se utilizaron conjuntos dentro de la función `gestion_reserva()` para clasificar los días de la semana y calcular el precio según el día de `check-in`. Se definieron los conjuntos `dias_semana_laboral` y `dias_fin_semana`, que mediante `.union()` forman el conjunto `todos_los_dias`. Luego, a partir del conjunto `dias_descuento`, se utilizó `.difference()` para obtener los días con aumento (`dias_aumento`), y `.intersection()` entre `dias_semana_laboral` y `dias_aumento` para identificar los días laborales de alta demanda (`dias_aumento_laboral`). Estas operaciones de conjuntos permiten determinar de forma clara y sin elementos repetidos qué descuento o recargo corresponde aplicar según el día generado aleatoriamente.

Pruebas Unitarias: Se utilizó el módulo `unittest` para validar el correcto funcionamiento de las funciones más importantes del sistema, separando cada prueba en un archivo independiente. `prueba_validaciones.py` comprueba que `validar_dni()` reconozca un DNI de 8 dígitos como válido. `prueba_altas.py` verifica que `cargar_matriz()` genere correctamente una matriz de 5 pisos. `prueba_búsquedas.py` controla que `habitaciones_hotel()` devuelva el tipo correcto de habitación al buscar por piso y número. Por último, `prueba_calculos.py` valida que `calcular_precio()` aplique correctamente el descuento del 20% para un día de semana con pago en efectivo. Cada prueba utiliza `assertEqual` o `assertTrue` para comparar el

resultado obtenido con el valor esperado, y se ejecutan mediante `python -m unittest` para confirmar que el sistema funciona como se espera antes de cada entrega.